1. AND Gate using ANN

AIM:

To implement an Artificial Neural Network (ANN) for the logical AND gate using a step activation function.

Description:

An Artificial Neural Network can be used to implement basic logical operations using an artificial neuron. In this experiment, a single neuron is used to implement the AND gate. The neuron takes two binary inputs x1 and x2, and each input is associated with a weight. The weights are set to w1 = 1 and w2 = 1, and the bias is set to b = -1.5. The weighted sum of the inputs is calculated using:

z = w1x1 + w2x2 + b

A binary step activation function is then applied. If z >= 0, the output is 1; otherwise, the output is 0. The neuron produces 1 only when both inputs are 1, which represents the behavior of an AND gate.

Code:

```
import pandas as pd

inputs = [
    (1, 1, 1),
    (1, 0, 0),
    (0, 1, 0),
    (0, 0, 0)
]

w1 = float(input("Enter Weight 1: "))
w2 = float(input("Enter Weight 2: "))

b = -1.5
L = 0.1

results = []

for x1, x2, gt in inputs:
    z = (x1 * w1) + (x2 * w2) + b

    if z >= 0:
        py = 1
    else:
        py = 0

    E = gt - py

    dw1 = L * E * x1
    dw2 = L * E * x2

    results.append([
        x1, x2, w1, w2, b, z, gt, py, E, L, dw1, dw2
    ])

columns = [
    "X1", "X2", "W1", "W2", "Bias",
    "Z", "GT", "PY", "E", "L", "ΔW1", "ΔW2"
]

df = pd.DataFrame(results, columns=columns)

print("\nAND GATE ANN")
print("=" * 100)
print(df.to_string(index=False))
print("=" * 100)
```

Output:

```
Enter Weight 1: 0.5
Enter Weight 2: -0.5

AND GATE ANN
====================================================================================================
```

```
 X1  X2  W1   W2  Bias    Z  GT  PY  E   L  ΔW1  ΔW2
  1   1 0.5 -0.5  -1.5 -1.5   1   0  1 0.1  0.1  0.1
  1   0 0.5 -0.5  -1.5 -1.0   0   0  0 0.1  0.0  0.0
  0   1 0.5 -0.5  -1.5 -2.0   0   0  0 0.1  0.0  0.0
  0   0 0.5 -0.5  -1.5 -1.5   0   0  0 0.1  0.0  0.0
==========================================================================================
```

2. OR Gate using ANN

AIM:

To implement an Artificial Neural Network (ANN) for the logical OR gate using a step activation function.

Description:

In this experiment, an artificial neuron is used to implement the logical OR gate. The neuron accepts two binary inputs x1 and x2. The weights are assigned as w1 = 1 and w2 = 1, with a bias of b = -0.5. The weighted sum is calculated as:

z = w1x1 + w2x2 + b

The step activation function is used to determine the final output. If z >= 0, the output is 1, and if z < 0, the output is 0. With these weights and bias, the neuron produces an output of 1 whenever at least one of the inputs is 1, thereby implementing the OR logic gate.

Code:

```
import pandas as pd

data = [
    [1, 1, 1],
    [1, 0, 1],
    [0, 1, 1],
    [0, 0, 0]
]

w1 = float(input("Enter Weight 1: "))
w2 = float(input("Enter Weight 2: "))

b = -0.5
learning_rate = 0.1

rows = []

for x1, x2, gt in data:

    z = (x1 * w1) + (x2 * w2) + b

    py = 1 if z >= 0 else 0

    error = gt - py

    dw1 = learning_rate * error * x1
    dw2 = learning_rate * error * x2

    dw = abs(dw1) + abs(dw2)

    rows.append([
        x1, x2, gt, py, error,
        learning_rate, dw, w1, w2
    ])

table = pd.DataFrame(
    rows,
    columns=["X1", "X2", "GT", "PY", "E", "L", "ΔW", "W1", "W2"]
)

print("\nOR GATE - ANN")
print("=" * 70)
print(table.to_string(index=False))
print("=" * 70)
```

Output:

```
Enter Weight 1: 0.5
Enter Weight 2: -0.5

OR GATE - ANN
======================================================================
 X1  X2  GT  PY  E   L  ΔW  W1   W2
  1   1   1   0  1 0.1 0.2 0.5 -0.5
  1   0   1   1  0 0.1 0.0 0.5 -0.5
  0   1   1   0  1 0.1 0.1 0.5 -0.5
  0   0   0   0  0 0.1 0.0 0.5 -0.5
======================================================================
```

3. Multilayer Perceptron using Backpropagation

AIM:

To implement a Multilayer Perceptron (MLP) using the backpropagation algorithm and observe the actual output, target, loss, and updated weights over 5 epochs.

Description:

A Multilayer Perceptron consists of an input layer, one or more hidden layers, and an output layer. In this experiment, a 2-2-1 MLP architecture is used, consisting of two input neurons, two hidden neurons, and one output neuron. The sigmoid activation function is used during forward propagation, and the backpropagation algorithm is used to update the weights based on the error. The network is trained for 5 epochs with a target output of 0.5 and a learning rate of 1. The actual output, loss, and updated weights are observed after each epoch.

Code:

```
import torch

x = torch.tensor([[1.0, 1.0]])

target = torch.tensor([[0.5]])

learning_rate = 1.0

epochs = 5

w1 = torch.tensor([
    [0.5, 0.5],
    [0.5, 0.5]
], requires_grad=True)

b1 = torch.tensor([[0.0, 0.0]], requires_grad=True)

w2 = torch.tensor([
    [0.5],
    [0.5]
], requires_grad=True)

b2 = torch.tensor([[0.0]], requires_grad=True)

results = []

for epoch in range(1, epochs + 1):

    hidden = torch.sigmoid(x @ w1 + b1)

    output = torch.sigmoid(hidden @ w2 + b2)

    loss = 0.5 * (target - output) ** 2

    loss.backward()

    with torch.no_grad():

        w1 -= learning_rate * w1.grad
        b1 -= learning_rate * b1.grad

        w2 -= learning_rate * w2.grad
        b2 -= learning_rate * b2.grad
```

```
    results.append([
        epoch,
        output.item(),
        target.item(),
        loss.item(),
        w1[0, 0].item(),
        w1[0, 1].item(),
        w1[1, 0].item(),
        w1[1, 1].item(),
        w2[0, 0].item(),
        w2[1, 0].item()
    ])

    w1.grad.zero_()
    b1.grad.zero_()
    w2.grad.zero_()
    b2.grad.zero_()

print("\nMLP USING BACKPROPAGATION")
print("Architecture: 2-2-1")
print("=" * 125)

print(
    f"{'Epoch':^7}"
    f"{'Actual Output':^16}"
    f"{'Target':^12}"
    f"{'Loss':^14}"
    f"{'W1(1,1)':^12}"
    f"{'W1(1,2)':^12}"
    f"{'W1(2,1)':^12}"
    f"{'W1(2,2)':^12}"
    f"{'W2(1)':^12}"
    f"{'W2(2)':^12}"
)

print("-" * 125)

for row in results:

    print(
        f"{row[0]:^7}"
        f"{row[1]:^16.6f}"
        f"{row[2]:^12.2f}"
        f"{row[3]:^14.6f}"
        f"{row[4]:^12.6f}"
        f"{row[5]:^12.6f}"
        f"{row[6]:^12.6f}"
        f"{row[7]:^12.6f}"
        f"{row[8]:^12.6f}"
        f"{row[9]:^12.6f}"
    )

print("=" * 125)
```

Output:

```
MLP USING BACKPROPAGATION
Architecture: 2-2-1
=============================================================================================================================
 Epoch  Actual Output     Target        Loss       W1(1,1)     W1(1,2)     W1(2,1)     W1(2,2)      W2(1)       W2(2)
-----------------------------------------------------------------------------------------------------------------------------
   1       0.675038        0.50       0.015319     0.496225    0.496225    0.496225    0.496225    0.471930    0.471930
   2       0.656901        0.50       0.012309     0.492927    0.492927    0.492927    0.492927    0.446157    0.446157
   3       0.639878        0.50       0.009783     0.490072    0.490072    0.490072    0.490072    0.422728    0.422728
   4       0.624127        0.50       0.007704     0.487619    0.487619    0.487619    0.487619    0.401612    0.401612
   5       0.609730        0.50       0.006020     0.485522    0.485522    0.485522    0.485522    0.382715    0.382715
=============================================================================================================================
```